
SmartRoom Manager

Réservation intelligente des salles d'un campus

Dossier technique complet

RÉALISÉ PAR

Dylan Menga Wanda — dylan.mengawanda@edu.ece.fr

Ngangne Takoudjou Angédarelle — angedarelle.ngangnetakoudjou@edu.ece.fr

Nguadjong Maeva — maevadior.nguadjong@edu.ece.fr

ÉTABLISSEMENT

ECE Paris — Troisième année du cycle ingénieur

Sommaire

1. Le projet en une page

1. Le problème posé
2. Ce que l'application fait
3. Le projet en chiffres

2. L'architecture

1. Deux applications, un contrat
2. Les quatre couches de l'API
3. Pourquoi le domaine est isolé

3. Le modèle de données

1. Les quarante-six entités
2. Ce que la base garantit elle-même
3. Les migrations

4. Les moteurs métier

1. Disponibilité
2. Règles de réservation
3. Recommandation
4. Conflits
5. Récurrence

5. L'API

1. 160 opérations en 15 domaines
2. Conventions
3. Authentification

6. L'interface

1. Les écrans de l'utilisateur
2. Les écrans de l'administration
3. Comment le front est organisé

7. Sécurité et données personnelles

1. Les mots de passe et les sessions
2. Les codes d'accès
3. Les permissions
4. Ce qui n'est jamais conservé

8. La qualité

1. 1 481 tests

2. Les trois familles de tests

3. Ce que les tests ont rattrapé

9. Le déploiement

10. Chronologie du projet

11. Limites assumées et suites possibles

12. Annexe — questions probables et réponses

1. Le projet en une page

De quoi il s'agit, et pourquoi cela demandait plus qu'un formulaire.

1.1 Le problème posé

Sur un campus, réserver une salle prend souvent plus de temps que la réunion elle-même. Quatre difficultés se cumulent, et aucune ne se résout par une simple liste :

- **On ne sait pas ce qui est libre.** On demande, on attend une réponse, on se déplace pour vérifier.
- **Deux personnes réservent le même créneau.** Sans arbitre, la salle revient au plus insistant.
- **Une salle réservée reste vide.** Rien ne la rend au suivant quand personne ne vient.
- **L'administration ne peut rien justifier.** Aucune trace des décisions, donc aucun arbitrage opposable.

1.2 Ce que l'application fait

Côté utilisateur

Un tunnel en quatre étapes — besoin, salle, créneau, confirmation — qui ne propose que des salles réellement libres *et* conformes aux règles en vigueur. Puis la vie de la réservation : la modifier, la partager, valider sa présence sur place avec un code d'accès, signaler un retard, l'annuler.

Côté administration

Le parc (bâtiments, étages, salles, équipements, plans de localisation), les règles de réservation par portée, les calendriers d'ouverture, les comptes et leurs permissions, l'arbitrage des conflits, le support, et un journal d'audit de chaque décision.

1.3 Le projet en chiffres

61 592 LIGNES HORS TESTS	1 481 TESTS	160 OPÉRATIONS D'API	48 TABLES
53 ROUTES D'INTERFACE	152 COMPOSANTS REACT	161 INDEX, DONT 51 PARTIELS	134 COMMITTS

Mesures relevées sur le dépôt le 1^{er} septembre 2026 : `wc -L` pour les lignes, le schéma OpenAPI pour les opérations, `information_schema` et `pg_indexes` pour la base, `git Log` pour les commits.

2. L'architecture

Deux applications séparées, reliées par un contrat explicite.

2.1 Deux applications, un contrat

Élément	Choix	Pourquoi
API	FastAPI, SQLAlchemy 2, Pydantic v2	Les schémas Pydantic produisent la documentation OpenAPI : le contrat ne peut pas mentir sur le code.
Base	PostgreSQL 16, extensions btree_gist, citext, pg_trgm, unaccent, vector, pgcrypto	Les types intervalle et l'exclusion GiST permettent d'interdire le chevauchement dans le moteur, pas dans le code.
Migrations	Alembic — 10 révisions	Le schéma se rejoue à l'identique sur n'importe quel poste.
Interface	React 18, Vite, Tailwind, React Router	Une seule page, un rendu rapide, et un thème sombre unique décliné en jetons de style.
Tests	pytest côté API, Vitest et Playwright côté interface	Les tests d'intégration tournent sur une vraie base : les contraintes ne se simulent pas.

2.2 Les quatre couches de l'API

app/	
domain/	les règles pures – ni base de données, ni HTTP availability.py conflicts.py recommendation.py rules.py organisation.py types.py
models/	les 46 entités SQLAlchemy et leurs contraintes
services/	ce que l'application fait – 16 services booking_service availability_service rules_service recommendation_service auth_service mail_service audit_service support_service stats_service ...
api/v1/	les routes et leurs schémas d'entrée-sortie 17 modules, 160 opérations

2.3 Pourquoi le domaine est isolé

La règle qu'on peut lire sans base de données

Les fonctions de `app/domain/` ne connaissent ni SQLAlchemy ni FastAPI : elles prennent des valeurs et rendent des valeurs. « Ce créneau respecte-t-il le préavis ? », « cette salle convient-elle à ce besoin ? » se testent alors sans base, en quelques millisecondes, et se relisent sans dérouler une requête SQL.

La conséquence pratique : quand une règle change, on modifie une fonction pure et son test, pas une route et son mock.

3. Le modèle de données

Quarante-six entités, et surtout ce que la base refuse d'elle-même.

3.1 Les quarante-six entités

Domaine	Entités
Identité	User, UserPreference, RefreshToken, PasswordResetToken
Administration	AdminAccount, PermissionGroup, Permission, AdminPermission, AdminInvitation, AdminInvitationPermission
Parc	Building, Floor, FloorPlan, Room, RoomPlacement, Equipment, RoomEquipment, RoomPhoto
Réservation	Booking, BookingParticipant, BookingEvent, BookingAccessCode, RecurrenceRule
Règles et ouverture	BookingRule, OpeningHour, ClosurePeriod, ClosureBuilding, ClosureRoom
Support	Ticket, TicketMessage, TicketResponseTemplate, AccessRequest
Connaissance	FaqCategory, FaqArticle, FaqFragment, FaqArticleLink, ChatbotIntent, ChatbotIntentKeyword
Assistant	ChatRole, ChatConversation, ChatMessage, ChatTour
Communication	Notification, EmailTemplate, EmailTemplateVariable
Traçabilité	AuditLog

3.2 Ce que la base garantit elle-même

110 contraintes CHECK ou EXCLUDE, 161 index dont 51 partiels, 18 énumérations. L'idée directrice : une règle qui doit toujours être vraie appartient au moteur, pas au code applicatif, parce que le code applicatif peut être contourné par un script, une migration, ou une seconde requête concurrente.

La contrainte la plus importante du projet

```
EXCLUDE USING gist (room_id WITH =, time_range WITH &&)
WHERE (status <> 'annulee' AND deleted_at IS NULL)
```

Deux réservations ne peuvent pas se chevaucher sur la même salle. Ce n'est pas une vérification : c'est une impossibilité. Deux requêtes simultanées qui demandent le même créneau ne peuvent pas gagner toutes les deux — PostgreSQL en refuse une, et l'application traduit ce refus en message.

Le `WHERE` compte autant que le reste : une réservation annulée ou supprimée ne bloque plus le créneau.

Quelques autres garanties

Contrainte	Ce qu'elle interdit
uq_admin_accounts_single_owner	Deux comptes propriétaires — il n'y a qu'un responsable ultime.
uq_booking_access_codes_active	Deux codes valables pour une même porte au même moment.
uq_booking_participants_organizer	Deux organisateurs pour une réservation.
uq_booking_rules_global / _building / _room	Deux jeux de règles pour la même portée.
ck_bookings_cancelled_not_checked_in	Annuler une réservation dont la présence est validée.
ck_access_requests_reference_format	Une référence de demande hors format #ABC-1234.

3.3 Les migrations

Dix révisions Alembic, de la création du schéma aux consignes libres d'administration. Chacune est réversible et nommée par son intention. Le schéma d'un poste neuf se reconstruit par `alembic upgrade head`, sans reprise manuelle.

4. Les moteurs métier

Cinq mécanismes qui font la différence avec un simple formulaire.

4.1 Disponibilité

Calculer ce qui est libre demande de croiser quatre sources : les heures d'ouverture (par salle, sinon par bâtiment, sinon globales), les périodes de fermeture, les réservations existantes, et le battement imposé entre deux réunions. Le résultat rendu à l'écran est *réservable tel quel* — pas « probablement libre ».

4.2 Règles de réservation

La résolution suit une cascade de portées, du plus précis au plus général :

règles de la salle → règles du bâtiment → règles globales

Chaque niveau ne surcharge que ce qu'il déclare. Les règles portent la durée minimale et maximale, le préavis, le quota hebdomadaire, la fenêtre de validation de présence, le seuil au-delà duquel une validation humaine devient nécessaire, et la consigne libre que l'administration souhaite afficher au moment de confirmer.

4.3 Recommandation

Un score sur 100 pondère six critères — capacité, équipements, accessibilité, proximité, taux d'occupation, préférences du compte — pour classer les salles éligibles. Le score n'élimine pas : il ordonne. Une salle qui ne convient pas n'est pas proposée du tout, ce qui relève de la disponibilité et des règles, pas de la recommandation.

4.4 Conflits

Quand la contrainte d'exclusion refuse une réservation, l'application ne renvoie pas l'utilisateur à zéro : elle propose des alternatives — même salle à un autre horaire, autre salle au même horaire — et, si l'utilisateur insiste, ouvre une demande d'accès exceptionnel que l'administration arbitre. Chaque arbitrage entre au journal d'audit.

4.5 Récurrence

Une réservation répétée engendre des occurrences *indépendantes* : annuler la séance du 12 n'annule pas celles du 19 et du 26. Ce choix évite la question insoluble du « que faire des exceptions » et rend chaque occurrence arbitable seule.

5. L'API

160 opérations sur 128 chemins, réparties en quinze domaines.

5.1 Répartition

Domaine	Opérations	Contenu
parc	33	Bâtiments, étages, salles, équipements, plans, photos
comptes	21	Profils, préférences, sessions, comptes d'administration, permissions
support	20	Tickets, messages, gabarits de réponse, base de connaissances
réservations	15	Cycle complet, participants, codes d'accès, validation de présence
statistiques	12	Occupation, absences, rapports d'administration
règles	11	Règles par portée, heures d'ouverture, fermetures
authentification	10	Connexion, rafraîchissement, Google, mot de passe oublié
notifications	10	Liste, lecture, préférences, gabarits de courriel
Assistant	8	Conversations, messages, visites guidées
demandes d'accès	5	Ouverture, arbitrage, alternatives
administration, audit, disponibilité, recommandation, technique	15	Journal, calendrier de disponibilité, classement des salles, santé

5.2 Conventions

- Les listes rendent une enveloppe `{ items, total, pagination }` : le nombre total ne se devine pas depuis la page reçue.
- Les énumérations circulent en minuscules anglaises ; la traduction est affaire d'affichage.
- Les dates sont en ISO 8601 UTC, sans exception.
- Une entrée malformée rend 422 avec un code nommé, jamais 500.
- Les erreurs suivent une forme unique : `{ error: { code, message } }`.

5.3 Authentification

Deux périmètres distincts. Le jeton utilisateur porte `scope=user` ; celui de l'administration `scope=admin`, émis par une route séparée. Un compte sans droits reçoit le même refus qu'un mot de passe faux — sans

quoi la route dirait qui est administrateur.

Les permissions ne sont pas déduites du jeton : elles sont relues à chaque reprise de session. C'est le seul moyen qu'une révocation prenne effet avant la prochaine connexion.

6. L'interface

53 routes, 52 pages, 152 composants — un seul thème, jamais deux versions d'un écran.

6.1 Les écrans de l'utilisateur

Écran	Rôle
Accueil	Prochaine réservation, recherche rapide, raccourcis
Tunnel de réservation	Besoin → salles éligibles → créneau → récapitulatif → confirmation
Mes réservations	Liste et calendrier, filtres, pagination
Détail d'une réservation	Code d'accès, plan, partage, modification, annulation
Validation de présence	Décompte, saisie du code, déclaration de retard
Explorer les salles	Filtres bâtiment, étage, capacité, équipements, accessibilité
Plan du bâtiment	Repérage visuel des salles
Notifications, profil, statistiques, centre d'aide, recherche globale	Espace personnel

6.2 Les écrans de l'administration

Écran	Rôle
Tableau de bord	Alertes, occupation, actions à traiter
Statistiques et rapports	Occupation, absences, export
Toutes les réservations	Vue globale, filtres, actions groupées
Conflits et demandes	Arbitrage, alternatives proposées
Parc	Bâtiments, salles, équipements, plans de localisation
Calendriers d'ouverture	Horaires et fermetures par portée
Règles de réservation	Globales et surcharges par salle
Utilisateurs, rôles et permissions	Annuaire, matrice permissions × comptes
Support et base de connaissances	Tickets, gabarits, articles
Modèles d'e-mails	Gabarits modifiables, variables déclarées
Journal d'audit	Qui a fait quoi, quand, et ce qui a changé

6.3 Comment le front est organisé

src/	
api/	le seul endroit qui connaît le transport HTTP
	adapters.js traduit les charges du serveur en objets d'écran
domain/	rien : les règles vivent côté serveur, et une seule fois
hooks/	useAsync, useDataTable, useFocusTrap, usePermission...
components/	152 composants, découpés par domaine
pages/	52 pages, un dossier par espace
e2e/	parcours complets, Playwright

Une seule page, jamais deux versions

Aucun écran n'a de variante « mobile » séparée. Les tableaux denses deviennent des cartes sous 1024 px, les grilles se replient, les cibles tactiles passent à 44 px — mais c'est le même composant. Deux versions divergent toujours : l'une reçoit une correction que l'autre attend.

7. Sécurité et données personnelles

Ce que le système protège, et comment il s'y prend.

7.1 Les mots de passe et les sessions

- Aucun mot de passe conservé : une empreinte **bcrypt**, qui ne permet pas de le retrouver.
- Le jeton d'accès vit **en mémoire**, jamais dans `localStorage` — un script injecté y lirait une session complète.
- La continuité entre deux chargements repose sur un cookie `httpOnly`, hors de portée du JavaScript.
- Le rafraîchissement fait tourner la famille de jetons ; un jeton joué invalide la famille entière.
- La connexion par compte Google vérifie le jeton d'identité côté serveur : signature RS256 contre les clés de Google, émetteur, **destinataire**, et adresse confirmée.

Pourquoi vérifier le destinataire

Sans le contrôle de l'audience, un jeton émis par Google pour *n'importe quelle autre application* ouvrirait une session ici. C'est la faute classique de l'authentification déléguée, et un test la verrouille explicitement.

7.2 Les codes d'accès

Un code d'accès ouvre une porte : il n'a pas de destinataire, et un message se transfère. Le traitement s'ensuit :

- Le code complet **n'existe qu'à l'instant de son émission**, dans le courriel qui le porte.
- La base n'en garde qu'une empreinte `bcrypt` et un indice — `E-****`.
- Les journaux techniques en sont expurgés : ils sont lus par plus de monde et conservés plus longtemps.
- Le partage d'une réservation ne l'emporte jamais, et l'écran le dit à l'utilisateur.
- Les notifications applicatives, écrites en base, n'en portent que l'indice.

7.3 Les permissions

Sept permissions réparties en groupes, accordées case par case à chaque compte d'administration. Le compte propriétaire les conserve toutes et ne peut pas être dégradé — lui retirer ses droits fermerait la configuration du système pour tout le monde. Chaque route protégée déclare la permission qu'elle exige ; le contrôle est fait côté serveur, l'interface ne fait que masquer ce qui serait refusé.

7.4 Ce qui n'est jamais conservé

- Les mots de passe en clair.
- Les codes d'accès complets, passé l'instant de l'émission.
- Les jetons dans les journaux.

- Aucun cookie de mesure d'audience ni traceur publicitaire : un seul cookie, technique et strictement nécessaire.

Une page de mentions légales décrit ce traitement en clair, et n'est pas recopiée d'un modèle : elle décrit le logiciel tel qu'il est écrit.

8. La qualité

1 481 tests, et la raison de chacun écrite à côté de lui.

1 026 TESTS API	455 TESTS INTERFACE	37 FICHIERS DE TEST API	41 FICHIERS DE TEST FRONT
---------------------------	-------------------------------	-----------------------------------	--

8.1 Les trois familles de tests

Tests de domaine

Rapides, sans base : ils exercent les fonctions pures — disponibilité, règles, recommandation, conflits. Une règle change, un test change.

Tests d'intégration

Sur une **vraie base PostgreSQL**. C'est une décision, pas une commodité : les contraintes d'exclusion, les index partiels et les requêtes de disponibilité ne se reproduisent fidèlement dans aucun double. Un test qui passerait sur SQLite ne dirait rien du comportement réel.

Tests de source

Quelques vérifications lisent le code source plutôt que le rendu : cibles tactiles à 44 px, coupure des textes longs, permission `min-w-0` sur les grilles. Elles existent parce que jsdom ne fait pas de mise en page : le défaut ne se lit qu'à l'écran, et une classe manquante ne manque à personne tant que personne ne regarde.

8.2 Ce que les tests ont rattrapé

Trois défauts réels, trouvés en fin de projet. Ils partagent une forme : **deux couches nommaient la même chose différemment, et rien ne le signalait.**

La matrice des permissions

L'écran qui gouverne qui peut quoi affichait ses vingt-huit cases vides alors que la base portait quinze droits accordés. La cause : la comparaison `admin.permissions.includes(permission.id)` confrontait une liste de *codes* (`rooms.manage`) à un *identifiant technique*. Jamais vrai. Le clic envoyait ce même identifiant à une route qui attend des codes : refusé.

Le journal d'audit

Le bloc « avant / après » — la raison d'être de l'écran — ne s'affichait jamais. L'adaptateur produisait `entry.before`, le composant lisait `entry.diff.before` : un niveau qui n'existe pas. L'optionnel absorbe l'absence, le repli donne un objet vide, la section se rend sans une ligne. Les 91 entrées portaient pourtant toutes leurs valeurs.

Les filtres de salles

Trois filtres sur cinq n'atteignaient jamais le serveur : l'écran envoyait `buildings`, `floors`, `accessible`, la fonction d'appel déstructurait `building`, `floor`, `accessibleOnly`. Trois `undefined` silencieux. Le plus grave : quelqu'un demandant une salle accessible PMR recevait tout le parc.

La leçon

Un champ absent d'une réponse JSON ne lève rien : il est simplement absent. Ces trois défauts ne produisaient aucune erreur, aucun avertissement, aucun test rouge — seulement un écran qui affichait faux. C'est pourquoi chaque correction du projet est accompagnée du test qui échouait avant elle : c'est la seule preuve que le défaut existait.

9. Le déploiement

De la machine de développement au domaine public.

9.1 En développement

```
docker compose up

db          PostgreSQL 16 + pgvector
api         FastAPI          → localhost:8000 (documentation sur /docs)
front       Vite             → localhost:5180
courriel    relais local     → localhost:8025
ollama      assistant local  → localhost:11434
```

Rien ne sort de la machine tant que `MAIL_ENABLED` vaut `false` : les courriels s'accumulent dans le relais local, où on les relit.

9.2 En production

- **Caddy** en frontal : certificat TLS automatique par Let's Encrypt, redirection du domaine secondaire.
- Front et API partagent une **origine unique**, ce qui permet au cookie de rester `SameSite=lax`.
- Les variables marquées « à remplacer » **font échouer le démarrage** plutôt que de laisser tourner un secret d'usine.
- Scripts de **sauvegarde et de restauration** de la base dans `deploy/`.
- Images étiquetées : laisser `latest` empêcherait de revenir en arrière.

9.3 Configuration

Tout passe par des variables d'environnement, décrites une par une dans `.env.example` avec la raison de leur valeur par défaut. Les principales : base de données, `JWT_SECRET`, relais de courrier, `GOOGLE_CLIENT_ID`, et `ORGANISATION_DOMAINS` qui distingue les comptes de l'établissement des adresses extérieures. Le fichier `.env` n'est pas suivi par git, et ne doit pas l'être.

10. Chronologie du projet

134 commits, du 20 août au 1^{er} septembre 2026.

Date	Étape
20 août	Espace utilisateur complet côté interface ; centre d'aide et page d'accueil.
22 août	Espace d'administration : seize écrans branchés sur les mêmes moteurs.
23 août	Modèle de données PostgreSQL et socle FastAPI. Moteur de disponibilité et API de réservation. Moteur de recommandation.
24 août	Refonte : domaine métier pur, API v1 unique, ancienne couche retirée.
fin août	Règles par portée, récurrence, support, audit, notifications, assistant. Déploiement, sauvegardes, chaîne d'intégration.
1 ^{er} sept.	Connexion Google, mentions légales, envoi réel des courriels. Campagne de vérification écran par écran et correction des défauts trouvés.

10.1 Ce que la fin du projet a apporté

La dernière journée n'a pas ajouté de fonctionnalité : elle a consisté à *parcourir chaque écran, en 375 px et en 1280 px, et à mesurer*. Cette campagne a produit une dizaine de corrections — écran de modification qui plantait, adresse d'un ticket qui n'ouvrait pas le ticket, cibles tactiles de 16 px, textes coupés, filtres inopérants, code d'accès conservé en clair dans les notifications.

Aucune de ces anomalies n'était visible dans le code. Toutes se voyaient à l'écran.

11. Limites assumées et suites possibles

Ce que nous n'avons pas fait, et pourquoi.

Limite	Raison
Authentification ADFS de l'école	SAML 2.0 exige une bibliothèque dédiée que nous nous étions interdit d'ajouter. La connexion Google remplit le même besoin.
Partage de fichiers depuis un navigateur de bureau	Brave et Firefox désactivent l'API de partage du système par choix de confidentialité. L'application le détecte, le retient, et bascule sur cinq applications cibles.
Application mobile installable	Le site est adapté au téléphone, mais n'est pas une application native ni une PWA installable.
Assistant de support	Fonctionne sur un modèle local ; il demanderait à être confronté à de vrais usages avant d'être élargi.
Aperçu Facebook des réservations	Le dialogue de Facebook ne transporte qu'une URL, et une réservation privée n'a pas de page publique. Le lien pointe la page d'accueil, et le résumé est copié.

11.1 Suites naturelles

- Ouvrir les filtres multiples à la recherche globale, comme sur l'exploration des salles.
- Étendre le journal d'audit aux actions des utilisateurs, pas seulement des administrateurs.
- Notifications poussées sur téléphone pour le rappel avant réunion.
- Export iCal d'un agenda complet, en plus de l'invitation par réservation.

12. Annexe — questions probables

Les questions qu'un jury pose, et de quoi y répondre.

« Comment empêchez-vous deux réservations sur le même créneau ? »

Par une contrainte d'exclusion GiST dans PostgreSQL, pas par une vérification applicative. Deux requêtes concurrentes ne peuvent pas gagner toutes les deux : le moteur en refuse une. L'application traduit ce refus en propositions d'alternatives.

« Où sont stockés les mots de passe ? »

Nulle part. La base contient une empreinte bcrypt, qui ne permet pas de retrouver le mot de passe. La réinitialisation passe par un jeton à usage unique et à durée limitée.

« Que se passe-t-il si quelqu'un réserve et ne vient pas ? »

La présence se valide sur place avec un code, dans une fenêtre qui s'ouvre au début du créneau et dure dix minutes par défaut — réglable par salle. Passé ce délai, une tâche planifiée libère la salle. L'utilisateur peut aussi signaler un retard, ce qui vaut validation et conserve la salle.

« Pourquoi tester sur une vraie base ? »

Parce que l'essentiel des garanties vit dans le moteur : exclusion sur intervalle, index partiels, contraintes de cohérence. Un double en mémoire les ignorerait, et les tests seraient verts sur un système cassé.

« Comment savez-vous que l'interface fonctionne vraiment ? »

Par des tests de composants, des parcours Playwright, et une campagne de mesure dans un vrai navigateur — largeurs de contrôle, débordements, défilement horizontal, erreurs de console — écran par écran, en 375 px et en 1280 px.

« Quelle est la partie dont vous êtes le plus fiers ? »

Le fait que les règles métier soient lisibles sans base de données : `app/domain/` contient des fonctions pures qu'on peut lire, tester et discuter sans dérouler une requête SQL. C'est ce qui a permis de corriger des règles en fin de projet sans casser le reste.

« Qu'est-ce qui vous a le plus surpris ? »

Que les défauts les plus graves aient été les plus silencieux. Un écran qui affiche zéro permission alors que la base en porte quinze ne lève aucune erreur : il faut le *regarder* pour s'en apercevoir.